I’m going to map the repo structure and detect the frontend/backend stack first, then I’ll measure complexity and churn so I can write the report with real evidence.I’m checking the manifests and top-level layout now to identify the frontend/backend stack and whether there’s existing complexity tooling I can rely on in the report.I’ve confirmed this is a mixed JavaScript/TypeScript repo with multiple app surfaces. Next I’m narrowing to the actual source trees so I can report on frontend and backend separately without counting vendored or generated files.I found the repo is centered on a JS/TS frontend stack, with a few app folders rather than a single backend service. I’m reading the root configs and then I’ll sample the main source trees to identify the highest-risk files for the report.I have the stack: React/Vite frontend(s) plus a Node/Express backend/proxy layer. I’m now measuring real hotspots from the local source trees and git history, while excluding `node_modules` and generated artifacts so the report stays grounded in actual application code.I’ve got enough structure to profile the main application areas. I’m now ranking candidate files by size and simple structural signals so the eventual report can focus on a few concrete hotspots instead of a generic repo sweep.The first pass hit an environment issue: this shell only has `python3`. I’m rerunning the same read-only analysis with the available interpreter so I can get the churn and size data.The size pass surfaced a few obvious hotspots: `cursor-agent-bridge/server/index.mjs`, `cursor-agent-bridge/server/jiraTwoWay.mjs`, and several oversized React/data files in `src` and `ADL-web/src`. I still need to confirm whether the git history is usable for churn metrics; if it is sparse, I’ll say so in the report rather than inventing churn numbers.I’ve got enough to pick representative hotspots. I’m reading the largest backend and frontend files directly now so I can capture real structure, not just line counts, for the evidence sections and diagrams.I’ve confirmed the code shape: `workflows.ts` is a giant frontend catalog, `IntentQuestionnaire.tsx` is a large stateful component, and the bridge/server files are the biggest backend hotspots. I’m collecting the last pieces of evidence now: line counts, commit churn for the hottest files, and which files were repeatedly involved in fix commits.I have enough to quantify ownership and churn, but I still need one more data point for defect-prone behavior. I’m checking which of the hot files appear in commit messages that reference fixes or bugs so the report can distinguish active maintenance from structural churn.I have the evidence needed to draft the implementation plan: the report will cover the main frontend surfaces in `src/` and `ADL-web/src/`, plus the Node bridge/backend in `cursor-agent-bridge/server/` and `client/*/src/`. I’m packaging that into a concise plan next so it can be reviewed before I write the discovery report.